

—— // VIMLRS // ——

vimlrs – Language Reference

v0.2.4

LANGUAGE REFERENCE

every builtin · command · option · function



MENKE
TECHNOLOGIES

Copyright and Attribution

vimlrs — VimL (Vimscript) in Rust, run outside the editor

Copyright © 2026 MenkeTechnologies. All rights reserved.

No part of this book may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the publisher, except for brief quotations in reviews and for the source excerpts identified below, which carry their own upstream licenses.

Published by MenkeTechnologies.

The trademarks and product names referenced in this book are the property of their respective owners. This book is an independent work: it is not affiliated with, authorized by, sponsored by, or otherwise endorsed by the Neovim project, the Vim project, their maintainers, the Rust Foundation, or any other project named in these pages.

The software this book describes

`vimlrs` is released under the MIT License, Copyright © 2026 MenkeTechnologies. The source is at github.com/MenkeTechnologies/vimlrs.

The upstream this book is a port of

`vimlrs` is a from-source port of **Neovim's C `eval/*` tree**, and those C sources are vendored in the `vimlrs` repository under `vendor/` as a read-only reference copy. Neovim is the work of its authors and contributors. Its `LICENSE.txt` states that "Neovim is licensed under the terms of the Apache 2.0 license, except for parts of Neovim that were contributed under the Vim license", and the Vim license text it carries begins "There are no restrictions on distributing unmodified copies of Vim except that they must include this license text."

Short excerpts of Neovim's C are quoted in this book for the purpose of commentary, criticism, and technical explanation — specifically, to show what a ported Rust function was validated against. Every such excerpt is cited to its file and function in `vendor/`.

The engine underneath

`vimlrs` executes on `fusevm` — the bytecode virtual machine and three-tier Cranelift JIT shared across the language fleet. `fusevm` is released under the MIT License, Copyright © 2026 MenkeTechnologies, and its source is at github.com/MenkeTechnologies/fusevm.



MT PUBLISHING



MT OPENSOURCE

Contents

Copyright and Attribution	1
String manipulation	3
List manipulation	10
Dictionary manipulation	13
Variables & Type	15
Number & Float	16
Files, Buffers & Lines	18
System & Environment	27
Functional & Misc	32
Ex Commands	56
Options	57
Special v: Variables	59

String manipulation

56 entries

byteidx

```
byteidx({string}, {nr} [, {utf16}])
```

Byte index of the {nr}'th character in {string}.

byteidxcomp

```
byteidxcomp({string}, {nr} [, {utf16}])
```

Like byteidx() but counting composing characters separately.

char2nr

```
char2nr({string} [, {utf8}])
```

Codepoint of the first character of {string}.

charclass

```
charclass({string})
```

Character class (0 white, 1 punctuation, 2 word, ...) of the first char of {string}.

charidx

```
charidx({string}, {idx} [, {countcc} [, {utf16}]])
```

Character index of byte {idx} in {string}.

digraph_get

```
digraph_get({chars})
```

The digraph character defined for the two-character {chars}.

digraph_getlist

```
digraph_getlist([{listall}])
```

List of [chars, digraph] pairs for the defined digraphs.

digraph_set

```
digraph_set({chars}, {digraph})
```

Define {chars} as a digraph producing {digraph}; returns v:true.

digraph_setlist

```
digraph_setlist({list})
```

Define multiple digraphs from a List of [chars, digraph] pairs.

escape

```
escape({string}, {chars})
```

{string} with each character in {chars} prefixed by a backslash.

expandcmd

```
expandcmd({string} [, {options}])
```

{string} with command-line special items ("%", '<cword>', ...) expanded.

fnameescape

```
fnameescape({string})
```

{string} escaped for use as a Vim command file-name argument.

gettext

```
gettext({text} [, {package}])
```

Translated {text} via gettext (identity when no catalog).

iconv

```
iconv({string}, {from}, {to})
```

{string} converted from encoding {from} to {to}.

join

```
join({list} [, {sep}])
```

String of {list} items joined by {sep} (default space).

keytrans

```
keytrans({string})
```

{string} with key codes translated to readable form (e.g. <C-A>).

list2str

```
list2str({list} [, {utf8}])
```

String built from the List of codepoint numbers {list}.

match

```
match({expr}, {pat} [, {start} [, {count}]])
```

Byte index where {pat} matches in {expr}, else -1.

matchbufline

```
matchbufline({buf}, {pat}, {lnum}, {end} [, {dict}])
```

List of matches of {pat} in buffer {buf} lines {lnum}..{end}.

matchend

```
matchend({expr}, {pat} [, {start} [, {count}]])
```

Byte index just after the match of {pat} in {expr}, else -1.

matchfuzzy

```
matchfuzzy({list}, {str} [, {dict}])
```

Items of {list} fuzzy-matching {str}, best first.

matchfuzzypos

```
matchfuzzypos({list}, {str} [, {dict}])
```

Like matchfuzzy() plus match positions and scores.

matchlist

```
matchlist({expr}, {pat} [, {start} [, {count}]])
```

List of the full match and submatches of {pat} in {expr}.

matchstr

```
matchstr({expr}, {pat} [, {start} [, {count}]])
```

Matched substring of {pat} in {expr}, or empty string.

matchstrlist

```
matchstrlist({list}, {pat} [, {dict}])
```

List of dicts describing matches of {pat} across the strings in {list}.

matchstrpos

```
matchstrpos({expr}, {pat} [, {start} [, {count}]])
```

[match, start, end] of {pat} in {expr}.

nr2char

```
nr2char({expr} [, {utf8}])
```

Character whose codepoint is {expr}.

printf

```
printf({fmt} [, {expr1} ...])
```

Formatted string from printf-style {fmt} and arguments.

repeat

```
repeat({expr}, {count})
```

{expr} (String or List) repeated {count} times.

sha256

```
sha256({string})
```

SHA-256 hex digest of {string}.

shellescape

```
shellescape({string} [, {special}])
```

{string} quoted for safe use as a shell command argument.

soundfold

```
soundfold({word})
```

Sound-folded (phonetic) form of {word}.

split

```
split({string} [, {pat} [, {keepempty}]])
```

List of {string} split on {pat} (default whitespace).

str2float

```
str2float({string} [, {quoted}])
```

Float parsed from {string}.

str2list

```
str2list({string} [, {utf8}])
```

List of codepoint numbers for each character in {string}.

str2nr

```
str2nr({string} [, {base}])
```

Leading integer parsed from {string} in {base} (default 10).

strcharlen

```
strcharlen({string})
```

Number of characters in {string}, ignoring composing characters.

strcharpart

```
strcharpart({src}, {start} [, {len} [, {skipcc}]])
```

Character substring of {src} from char {start} for {len} chars.

strchars

```
strchars({string} [, {skipcc}])
```

Number of characters in {string}, counting composing chars unless {skipcc}.

strdisplaywidth

```
strdisplaywidth({string} [, {col}])
```

Display cell width of {string}, honoring 'tabstop'.

strgetchar

```
strgetchar({str}, {index})
```

Codepoint at char {index} of {str}, or -1.

stridx

```
stridx({haystack}, {needle} [, {start}])
```

Byte index of first {needle} in {haystack} at/after {start}, else -1.

string

```
string({expr})
```

String representation of {expr}, quoting embedded strings (inverse of eval()).

strlen

```
strlen({string})
```

Number of bytes in {string}.

strpart

```
strpart({src}, {start} [, {len} [, {chars}]])
```

Byte (or char) substring of {src} from {start} for {len}.

stridx

```
stridx({haystack}, {needle} [, {start}])
```

Byte index of last {needle} in {haystack} at/before {start}, else -1.

strtrans

```
strtrans({string})
```

{string} with unprintable characters shown as ^X / <xx>.

strutf16len

```
strutf16len({string} [, {countcc}])
```

Number of UTF-16 code units in {string}.

strwidth

```
strwidth({string})
```

Display cell width of {string} (no tab expansion).

submatch

```
submatch({nr} [, {list}])
```

The {nr}'th submatch of the substitute() replacement.

substitute

```
substitute({string}, {pat}, {sub}, {flags})
```

{string} with matches of {pat} replaced by {sub}; {flags} 'g' replaces all.

tolower

```
tolower({string})
```

{string} with all letters lowercased.

toupper

```
toupper({string})
```

{string} with all letters uppercased.

tr

```
tr({src}, {fromstr}, {tostr})
```

{src} with each char in {fromstr} replaced by the matching char in {tostr}.

trim

```
trim({text} [, {mask} [, {dir}]])
```

{text} with leading/trailing {mask} chars (default whitespace) removed.

utf16idx

```
utf16idx({string}, {idx} [, {countcc} [, {charidx}]])
```

UTF-16 code-unit index of byte {idx} in {string}.

List manipulation

22 entries

add

```
add({object}, {expr})
```

Append {expr} to List or Blob {object}; returns {object}.

blob2list

```
blob2list({blob})
```

List of byte-value Numbers for each byte of {blob}.

copy

```
copy({expr})
```

Shallow copy of {expr}.

count

```
count({comp}, {expr} [, {ic} [, {start}]])
```

Number of times {expr} occurs in String/List/Dict {comp}.

deepcopy

```
deepcopy({expr} [, {noref}])
```

Recursive (deep) copy of {expr}.

flatten

```
flatten({list} [, {maxdepth}])
```

Flatten nested List {list} in place up to {maxdepth}.

flattennew

```
flattennew({list} [, {maxdepth}])
```

Flattened copy of nested List {list}, leaving {list} unchanged.

foreach

```
foreach({expr1}, {expr2})
```

Apply {expr2} to each item of {expr1} for its side effects; returns {expr1}.

index

```
index({object}, {expr} [, {start} [, {ic}]]])
```

Index of first item equal to {expr} in {object}, else -1.

indexof

```
indexof({object}, {expr} [, {opts}])
```

Index of first item for which {expr} is true, else -1.

insert

```
insert({object}, {item} [, {idx}])
```

Insert {item} into List/Blob {object} before index {idx} (default 0).

list2blob

```
list2blob({list})
```

Blob built from the List of byte-value Numbers {list}.

mapnew

```
mapnew({expr1}, {expr2})
```

Like map() but returns a new container, leaving {expr1} unchanged.

max

```
max({expr})
```

Maximum value among the items of List/Dict {expr} (0 if empty).

min

```
min({expr})
```

Minimum value among the items of List/Dict {expr} (0 if empty).

range

```
range({expr} [, {max} [, {stride}]]])
```

List of numbers from {expr} to {max} stepping by {stride}.

reduce

```
reduce({object}, {func} [, {initial}])
```

Fold {object} to a single value via {func}, starting at {initial}.

remove

```
remove({object}, {idx} [, {end}])
```

Remove and return item {idx} (or range {idx}..{end}) from {object}.

reverse

```
reverse({object})
```

Reverse List/Blob {object} in place; returns {object}.

slice

```
slice({expr}, {start} [, {end}])
```

Sub-list/Blob/String of {expr} from {start} up to (not including) {end}.

sort

```
sort({list} [, {how} [, {dict}])
```

Sort {list} in place; {how} sets comparison; returns {list}.

uniq

```
uniq({list} [, {how} [, {dict}])
```

Remove adjacent duplicate items from sorted {list} in place.

Dictionary manipulation

9 entries

dictwatcheradd

```
dictwatcheradd({dict}, {pattern}, {callback})
```

Register {callback} to fire when keys matching {pattern} in {dict} change.

dictwatcherdel

```
dictwatcherdel({dict}, {pattern}, {callback})
```

Remove a watcher previously added with dictwatcheradd().

extend

```
extend({expr1}, {expr2} [, {expr3}])
```

Append/merge {expr2} into List/Dict {expr1} in place; returns {expr1}.

extendnew

```
extendnew({expr1}, {expr2} [, {expr3}])
```

Like extend() but returns a new container, leaving {expr1} unchanged.

get

```
get({collection}, {key} [, {default}])
```

Item {key}/{idx} of Dict/List/Blob, or {default} if absent.

has_key

```
has_key({dict}, {key})
```

1 if {dict} has an entry {key}, else 0.

items

```
items({dict})
```

List of [key, value] pairs in {dict}.

keys

```
keys({dict})
```

List of all keys in {dict}.

values

```
values({dict})
```

List of all values in {dict}.

Variables & Type

8 entries

empty

```
empty({expr})
```

1 if {expr} is empty (0, "", [], {}, v:null), else 0.

exists

```
exists({expr})
```

1 if variable/option/function/command {expr} exists, else 0.

funcref

```
funcref({name} [, {arglist}] [, {dict}])
```

Funcref bound by reference to {name} (ignores later redefinition).

function

```
function({name} [, {arglist}] [, {dict}])
```

Funcref for builtin/user function {name}, optionally bound to args/dict.

garbagecollect

```
garbagecollect([{atexit}])
```

Run the cyclic-reference garbage collector (stub in standalone).

islocked

```
islocked({expr})
```

1 if the variable named by {expr} is locked, 0 if unlocked, -1 if it does not exist.

len

```
len({expr})
```

Length of String/List/Dict/Blob {expr}.

type

```
type({expr})
```

Numeric type code of {expr} (0 Number, 1 String, 2 Func, 3 List, 4 Dict, ...).

Number & Float

13 entries

abs

```
abs({expr})
```

Absolute value of {expr} (Float in -> Float out).

and

```
and({expr1}, {expr2})
```

Bitwise AND of {expr1} and {expr2}.

atan2

```
atan2({expr1}, {expr2})
```

Arc tangent of {expr1}/{expr2} in radians (-pi..pi).

float2nr

```
float2nr({expr})
```

{expr} truncated toward zero to a Number.

fmod

```
fmod({expr1}, {expr2})
```

Floating-point remainder of {expr1} / {expr2}.

invert

```
invert({expr})
```

Bitwise complement (NOT) of {expr}.

isinf

```
isinf({expr})
```

1 if {expr} is +inf, -1 if -inf, else 0.

isnan

```
isnan({expr})
```

1 if {expr} is a NaN Float, else 0.

or

```
or({expr1}, {expr2})
```

Bitwise OR of {expr1} and {expr2}.

pow

```
pow({x}, {y})
```

{x} raised to the power {y} as a Float.

rand

```
rand([{expr}])
```

Pseudo-random 32-bit Number, optionally from seed-list {expr}.

srand

```
srand([{expr}])
```

Initialize and return a seed List for rand().

xor

```
xor({expr1}, {expr2})
```

Bitwise XOR of {expr1} and {expr2}.

Files, Buffers & Lines

77 entries

append

```
append({lnum}, {text})
```

Append {text} (String or List) below line {lnum}; 0 on success.

appendbufline

```
appendbufline({buf}, {lnum}, {text})
```

Append {text} below line {lnum} of buffer {buf}.

browse

```
browse({save}, {title}, {initdir}, {default})
```

Prompt for a file name via a dialog (GUI).

browsedir

```
browsedir({title}, {initdir})
```

Prompt for a directory via a dialog (GUI).

bufadd

```
bufadd({name})
```

Add a buffer for file {name} (without loading) and return its number (stub in standalone).

bufexists

```
bufexists({buf})
```

1 if buffer {buf} exists, else 0 (stub in standalone).

buflisted

```
buflisted({buf})
```

1 if buffer {buf} exists and is listed, else 0 (stub in standalone).

bufload

```
bufload({buf})
```

Ensure buffer {buf} is loaded (reading the file if needed) (stub in standalone).

bufloaded

```
bufloaded({buf})
```

1 if buffer {buf} exists and is loaded, else 0 (stub in standalone).

bufname

```
bufname([ {buf} ])
```

Name of buffer {buf} (default current) (stub in standalone).

bufnr

```
bufnr([ {buf} [, {create}] ])
```

Buffer number of {buf}; optionally create it (stub in standalone).

bufwinid

```
bufwinid({buf})
```

window-ID of the first window showing buffer {buf}, else -1 (stub in standalone).

bufwinnr

```
bufwinnr({buf})
```

Window number of the first window showing buffer {buf}, else -1 (stub in standalone).

byte2line

```
byte2line({byte})
```

Line number containing byte offset {byte}, else -1.

changenr

```
changenr()
```

Sequence number of the current entry in the buffer's change list (stub in standalone).

charcol

```
charcol({expr} [, {winid}])
```

Character column for position {expr}.

chdir

```
chdir({dir})
```

Change the working directory to {dir}; returns the previous directory.

cindent

```
cindent({lnum})
```

C-indent amount Vim would use for line {lnum}.

col

```
col({expr} [, {winid}])
```

Byte column for position {expr}.

delete

```
delete({fname} [, {flags}])
```

Delete file (or, with 'd/rf', directory) {fname}; 0 on success.

deletebufline

```
deletebufline({buf}, {first} [, {last}])
```

Delete lines {first}..{last} from buffer {buf}.

executable

```
executable({expr})
```

1 if {expr} is an executable found in \$PATH, else 0.

exepath

```
exepath({expr})
```

Full path of the executable {expr}, or empty if not found.

expand

```
expand({string} [, {nosuf} [, {list}]])
```

Expand wildcards and keywords ('%', '<cfile>', '~', ...) in {string}.

filecopy

```
filecopy({from}, {to})
```

Copy file {from} to {to}; v:true on success.

filereadable

```
filereadable({file})
```

1 if {file} exists and is readable, else 0.

filewritable

```
filewritable({file})
```

1 if {file} is writable, 2 if it is a writable directory, else 0.

finddir

```
finddir({name} [, {path} [, {count}]])
```

Find directory {name} in {path}; the {count}'th match (or List).

findfile

```
findfile({name} [, {path} [, {count}]])
```

Find file {name} in {path}; the {count}'th match (or List).

fnamemodify

```
fnamemodify({fname}, {mods})
```

Modify file name {fname} per the :p/:h/:t/:r/:e {mods}.

getbufinfo

```
getbufinfo([{buf}])
```

List of dicts with information about buffers.

getbufline

```
getbufline({buf}, {lnum} [, {end}])
```

List of lines {lnum}..{end} from buffer {buf}.

getbufoneline

```
getbufoneline({buf}, {lnum})
```

Single line {lnum} from buffer {buf} as a String.

getbufvar

```
getbufvar({buf}, {varname} [, {def}])
```

Buffer-local variable/option {varname} from buffer {buf}.

getchangelist

```
getchangelist([{buf}])
```

[changelist, index] for buffer {buf}.

getcwd

```
getcwd([ {winnr} [, {tabnr} ] ])
```

Current working directory (optionally of a window/tab).

getfontname

```
getfontname([ {name} ])
```

Name of the font currently in use (GUI) (stub in standalone).

getfperm

```
getfperm({fname})
```

Permissions of {fname} as a 9-character 'rwxrwxrwx' String.

getfsize

```
getfsize({fname})
```

Size of file {fname} in bytes (-1 if absent, -2 if too large).

getftime

```
getftime({fname})
```

Last modification time of {fname} as seconds since the epoch, else -1.

getftype

```
getftype({fname})
```

Type of file {fname}: 'file', 'dir', 'link', 'fifo', etc.

getline

```
getline({lnum} [, {end} ])
```

Text of line {lnum} (or List of lines {lnum}..{end}) in the current buffer.

glob

```
glob({expr} [, {nosuf} [, {list} [, {alllinks} ] ] ])
```

Expand file wildcards {expr}; String (or List) of matching paths.

glob2regpat

```
glob2regpat({string})
```

Vim regexp pattern equivalent to the file glob {string}.

globpath

```
globpath({path}, {expr} [, {nosuf} [, {list} [, {alllinks}]]])
```

Expand {expr} in every directory of {path}.

haslocaldir

```
haslocaldir([{winnr} [, {tabnr}]]])
```

1 if the window/tab has a local working directory.

indent

```
indent({lnum})
```

Indent (in spaces) of line {lnum} in the current buffer.

isabsolutepath

```
isabsolutepath({path})
```

1 if {path} is an absolute path, else 0.

isdirectory

```
isdirectory({directory})
```

1 if {directory} exists and is a directory, else 0.

last_buffer_nr

```
last_buffer_nr()
```

Number of the last (highest-numbered) buffer (stub in standalone).

line

```
line({expr} [, {winid}])
```

Line number for position {expr} (., '\$', mark, ...).

line2byte

```
line2byte({lnum})
```

Byte offset of the first byte of line {lnum} (1-based), else -1.

lispindent

```
lispindent({lnum})
```

Lisp indent amount Vim would use for line {lnum}.

mkdir

```
mkdir({name} [, {flags} [, {prot}]])
```

Create directory {name} ('p' makes parents); 1 on success.

nextnonblank

```
nextnonblank({lnum})
```

Line number of the first non-blank line at/after {lnum}, else 0.

pathshorten

```
pathshorten({path} [, {len}])
```

Shorten directory components of {path} to {len} characters.

prevnonblank

```
prevnonblank({lnum})
```

Line number of the first non-blank line at/before {lnum}, else 0.

readblob

```
readblob({fname} [, {offset} [, {size}]])
```

Blob of the (partial) contents of file {fname}.

readdir

```
readdir({directory} [, {expr}])
```

List of file names in {directory}, optionally filtered by {expr}.

readfile

```
readfile({fname} [, {type} [, {max}]])
```

List of lines read from file {fname}.

rename

```
rename({from}, {to})
```

Rename/move file {from} to {to}; 0 on success.

resolve

```
resolve({filename})
```

Resolve symbolic links in {filename} to its real path.

setbufline

```
setbufline({buf}, {lnum}, {text})
```

Set line {lnum} of buffer {buf} to {text}.

setbufvar

```
setbufvar({buf}, {varname}, {val})
```

Set buffer-local variable/option {varname} in buffer {buf} (stub in standalone).

setfperm

```
setfperm({fname}, {mode})
```

Set permissions of {fname} from a 9-character 'rwxrwxrwx' {mode}.

setline

```
setline({lnum}, {text})
```

Set line {lnum} to {text} (String or List of lines); 0 on success.

simplify

```
simplify({filename})
```

Simplify {filename} by resolving '.', '..' and doubled separators (no disk access).

swapfilelist

```
swapfilelist()
```

List of swap files found in the 'directory' directories.

swapinfo

```
swapinfo({fname})
```

Dict of information read from swap file {fname}.

swapname

```
swapname({buf})
```

Swap-file name of buffer {buf}, or empty if none (stub in standalone).

tempname

```
tempname()
```

Name of a unique, not-yet-existing temporary file.

undofile

```
undofile({name})
```

Undo-file path Vim would use for file {name}.

undotree

```
undotree([{buf}])
```

Dict describing the undo tree of the buffer.

virtcol

```
virtcol({expr} [, {list} [, {winid}]])
```

Screen (virtual) column of position {expr}.

virtcol2col

```
virtcol2col({winid}, {lnum}, {col})
```

Byte index in line {lnum} of screen column {col} in window {winid}.

winbufnr

```
winbufnr({nr})
```

Buffer number of window {nr}, else -1 (stub in standalone).

writefile

```
writefile({object}, {fname} [, {flags}])
```

Write List of lines or Blob {object} to file {fname}; 0 on success.

System & Environment

44 entries

chanclose

```
chanclose({id} [, {stream}])
```

Close channel {id} (or one of its streams) (stub in standalone).

chansend

```
chansend({id}, {data})
```

Send {data} to channel {id}; returns bytes written (stub in standalone).

environ

```
environ()
```

Dict of all environment variables and their values.

getenv

```
getenv({name})
```

Value of environment variable {name}, or v:null if unset.

getpid

```
getpid()
```

Process ID of the Vim process.

hostname

```
hostname()
```

Name of the host Vim is running on.

jobpid

```
jobpid({id})
```

Process ID of job {id} (stub in standalone).

jobresize

```
jobresize({id}, {width}, {height})
```

Resize the pseudo-terminal of job {id} (stub in standalone).

jobstart

```
jobstart({cmd} [, {opts}])
```

Start job running {cmd}; returns a job id (or <=0 on failure) (stub in standalone).

jobstop

```
jobstop({id})
```

Stop job {id} (stub in standalone).

jobwait

```
jobwait({ids} [, {timeout}])
```

Wait for the jobs {ids} to finish; List of exit codes.

json_decode

```
json_decode({string})
```

Vim value decoded from JSON text {string}.

json_encode

```
json_encode({expr})
```

JSON text encoding of {expr}.

libcall

```
libcall({lib}, {func}, {arg})
```

Call function {func} in dynamic library {lib} (String result) (stub in standalone).

libcallnr

```
libcallnr({lib}, {func}, {arg})
```

Call function {func} in dynamic library {lib} (Number result) (stub in standalone).

localtime

```
localtime()
```

Current time as seconds since the Unix epoch.

luaeval

```
luaeval({expr} [, {expr}])
```

Evaluate Lua expression {expr} and return the result.

msgpackdump

```
msgpackdump({list} [, {type}])
```

msgpack byte representation of the items in {list}.

msgpackparse

```
msgpackparse({data})
```

List of Vim values parsed from msgpack {data}.

perlval

```
perlval({expr})
```

Evaluate Perl expression {expr} and return the result.

py3eval

```
py3eval({expr})
```

Evaluate Python 3 expression {expr} and return the result.

pyeval

```
pyeval({expr})
```

Evaluate Python (2) expression {expr} and return the result.

pyxeval

```
pyxeval({expr})
```

Evaluate Python (pyx) expression {expr} and return the result.

reltime

```
reltime([{{start}} [, {{end}}]])
```

High-resolution time value, or an elapsed interval.

reltimefloat

```
reltimefloat({time})
```

reltime() value as a Float number of seconds.

reltimestr

```
reltimestr({time})
```

reltime() value formatted as a 'seconds.microseconds' String.

rpcnotify

```
rpcnotify({channel}, {event} [, {args} ...])
```

Send an RPC notification {event} to {channel} (stub in standalone).

rpcrequest

```
rpcrequest({channel}, {method} [, {args} ...])
```

Send a blocking RPC request {method} to {channel} and return the result (stub in standalone).

rpcstart

```
rpcstart({prog} [, {argv}])
```

Deprecated: start an RPC channel to {prog} (stub in standalone).

rpcstop

```
rpcstop({channel})
```

Deprecated: close RPC channel {channel} (stub in standalone).

rubyeval

```
rubyeval({expr})
```

Evaluate Ruby expression {expr} and return the result.

serverlist

```
serverlist()
```

Newline-separated list of server addresses Vim is listening on.

serverstart

```
serverstart([[{address}])
```

Start an RPC server, returning its listen address (stub in standalone).

serverstop

```
serverstop({address})
```

Stop the RPC server listening on {address} (stub in standalone).

setenv

```
setenv({name}, {val})
```

Set environment variable {name} to {val} (v:null unsets it).

sockconnect

```
sockconnect({mode}, {address} [, {opts}])
```

Connect a socket to {address} (tcp/pipe); returns a channel id (stub in standalone).

stdioopen

```
stdioopen({opts})
```

Open a channel on Vim's own stdin/stdout (stub in standalone).

stdpath

```
stdpath({what})
```

Standard config/data/cache/state path(s) for {what}.

strftime

```
strftime({format} [, {time}])
```

Format time {time} (default now) with strftime() {format}.

strptime

```
strptime({format}, {timestring})
```

Parse {timestring} per {format} into seconds since the epoch.

system

```
system({cmd} [, {input}])
```

Run shell {cmd}, returning its output as a String.

systemlist

```
systemlist({cmd} [, {input}])
```

Run shell {cmd}, returning its output as a List of lines.

termopen

```
termopen({cmd} [, {opts}])
```

Open a terminal running {cmd} in the current buffer (stub in standalone).

windowsversion

```
windowsversion()
```

Microsoft Windows version string (" on other systems) (stub in standalone).

Functional & Misc

212 entries

api_info

```
api_info()
```

Dict describing the Nvim API (version, functions, types).

argc

```
argc([[winid]])
```

Number of files in the argument list.

argidx

```
argidx()
```

Current index into the argument list.

arglistid

```
arglistid([winnr] [, {tabnr}])
```

Identifier of the argument list used by a window.

argv

```
argv([nr] [, {winid}])
```

Argument-list entry {nr} (or the whole List).

assert_equal

```
assert_equal({expected}, {actual} [, {msg}])
```

Add an error to v:errors unless {expected} == {actual}.

assert_equalfile

```
assert_equalfile({fname1}, {fname2} [, {msg}])
```

Add an error to v:errors unless the two files have equal contents.

assert_exception

```
assert_exception({error} [, {msg}])
```

Add an error to v:errors unless v:exception contains {error}.

assert_false

```
assert_false({actual} [, {msg}])
```

Add an error to v:errors unless {actual} is false.

assert_inrange

```
assert_inrange({lower}, {upper}, {actual} [, {msg}])
```

Add an error to v:errors unless {lower} <= {actual} <= {upper}.

assert_match

```
assert_match({pat}, {actual} [, {msg}])
```

Add an error to v:errors unless {pat} matches {actual}.

assert_notequal

```
assert_notequal({expected}, {actual} [, {msg}])
```

Add an error to v:errors unless {expected} != {actual}.

assert_notmatch

```
assert_notmatch({pat}, {actual} [, {msg}])
```

Add an error to v:errors unless {pat} does not match {actual}.

assert_report

```
assert_report({msg})
```

Add {msg} to v:errors unconditionally.

assert_true

```
assert_true({actual} [, {msg}])
```

Add an error to v:errors unless {actual} is true.

call

```
call({func}, {arglist} [, {dict}])
```

Call {func} with the List {arglist} as its arguments (stub in standalone).

clearmatches

```
clearmatches([win])
```

Clear all matches added with matchadd()/matchaddpos().

cmdcomplete_info

```
cmdcomplete_info([{what}])
```

Dict of information about the current command-line completion.

complete

```
complete({startcol}, {matches})
```

Set the Insert-mode completion list to {matches} starting at {startcol} (stub in standalone).

complete_add

```
complete_add({expr})
```

Add {expr} to the completion list during a 'completefunc'.

complete_check

```
complete_check()
```

1 if completion was aborted by typed input.

complete_info

```
complete_info([{what}])
```

Dict of information about the current Insert-mode completion.

confirm

```
confirm({msg} [, {choices} [, {default} [, {type}]]])
```

Show dialog {msg} with {choices}; return the chosen button number.

ctxget

```
ctxget([{index}])
```

Dict for the {index}'th entry on the context stack.

ctxpop

```
ctxpop()
```

Pop and restore the top editor context from the stack (stub in standalone).

ctxpush

```
ctxpush([{types}])
```

Push the editor context (of {types}) onto the context stack (stub in standalone).

ctxset

```
ctxset({context} [, {index}])
```

Set the {index}'th context-stack entry from {context} (stub in standalone).

ctxsize

```
ctxsize()
```

Number of entries on the context stack (stub in standalone).

cursor

```
cursor({lnum}, {col} [, {off}])
```

Move the cursor to line {lnum}, column {col}.

debugbreak

```
debugbreak({pid})
```

Interrupt process {pid} (MS-Windows debugging helper) (stub in standalone).

did_filetype

```
did_filetype()
```

1 if a FileType autocommand has run for the current buffer (stub in standalone).

diff_filler

```
diff_filler({lnum})
```

Number of filler lines above diff line {lnum}.

diff_hlID

```
diff_hlID({lnum}, {col})
```

Diff highlight ID at line {lnum}, column {col}.

eval

```
eval({string})
```

Evaluate {string} as a Vim expression (inverse of string()) (stub in standalone).

eventhandler

```
eventhandler()
```

1 if Vim is inside an event handler (e.g. processing a callback) (stub in standalone).

execute

```
execute({command} [, {silent}])
```

Execute Ex {command}(s) and return their captured output (stub in standalone).

feedkeys

```
feedkeys({string} [, {mode}])
```

Push {string} into the typeahead buffer as if typed (stub in standalone).

filter

```
filter({expr1}, {expr2})
```

Keep items of {expr1} for which {expr2} is non-zero; in place.

foldclosed

```
foldclosed({lnum})
```

First line of the closed fold at {lnum}, else -1.

foldclosedend

```
foldclosedend({lnum})
```

Last line of the closed fold at {lnum}, else -1.

foldlevel

```
foldlevel({lnum})
```

Fold nesting level of line {lnum}.

foldtext

```
foldtext()
```

Text Vim shows for the closed fold on the current line.

foldtextresult

```
foldtextresult({lnum})
```

The 'foldtext' string for the fold at {lnum}.

foreground

```
foreground()
```

Bring the Vim window to the foreground (GUI/clients) (stub in standalone).

fullcommand

```
fullcommand({name} [, {vim9}])
```

Full Ex-command name for the abbreviation {name}.

getcellwidths

```
getcellwidths()
```

List of [low, high, width] cell-width overrides set with setcellwidths().

getchar

```
getchar([{expr} [, {opts}])
```

Get one character of user input as a Number or special-key String.

getcharmod

```
getcharmod()
```

Bitmask of modifier keys held for the last getchar().

getcharpos

```
getcharpos({expr})
```

Like getpos() but the column is a character index.

getcharsearch

```
getcharsearch()
```

Dict describing the last character search (f/t/F/T).

getcharstr

```
getcharstr([{expr} [, {opts}])
```

Like getchar() but always returns the character as a String.

getcmlpat

```
getcmlpat()
```

Pattern used for completion of the current command line.

getcmltype

```
getcmltype()
```

Completion type of the current command line.

getcmdline

```
getcmdline()
```

Current command-line contents.

getcmdpos

```
getcmdpos()
```

1-based byte position of the cursor in the command line.

getcmdprompt

```
getcmdprompt()
```

Prompt string of the current input()/command line.

getcmdscreenpos

```
getcmdscreenpos()
```

Screen cursor position within the command line.

getcmdtype

```
getcmdtype()
```

Type of the current command line (':', '/', '?', '@', '-', '=').

getcmdwintype

```
getcmdwintype()
```

Type of the command-line window, or empty if none (stub in standalone).

getcompletion

```
getcompletion({pat}, {type} [, {filtered}])
```

List of command-line completion candidates for {pat} of {type}.

getcompletiontype

```
getcompletiontype({pat})
```

Completion type Vim would use for command line {pat}.

getcurpos

```
getcurpos([winid])
```

Cursor position of a window including curswant.

getcursorcharpos

```
getcursorcharpos([winid])
```

Like getcurpos() but the column is a character index.

getjumplist

```
getjumplist([winnr] [, {tabnr}])
```

[jumplist, index] for a window.

getloclist

```
getloclist({nr} [, {what}])
```

List of location-list entries for window {nr}.

getmarklist

```
getmarklist([buf])
```

List of marks (global, or local to buffer {buf}).

getmatches

```
getmatches([win])
```

List of matches added with matchadd()/setmatches().

getmousepos

```
getmousepos()
```

Dict with the last known mouse position.

getpos

```
getpos({expr})
```

Position [bufnum, lnum, col, off] of {expr}.

getqflist

```
getqflist([what])
```

List of quickfix entries (or info per {what}).

getreg

```
getreg([regname] [, {list} [, {opts}]]])
```

Contents of register {regname}.

getreginfo

```
getreginfo([ {regname} ])
```

Dict describing register {regname} (value, type, width).

getregion

```
getregion({pos1}, {pos2} [, {opts}])
```

List of text lines for the region between {pos1} and {pos2}.

getregionpos

```
getregionpos({pos1}, {pos2} [, {opts}])
```

List of [start, end] position pairs for each line of a region.

getregtype

```
getregtype([ {regname} ])
```

Type of register {regname}: 'v', 'V', or blockwise.

getscriptinfo

```
getscriptinfo([ {opts} ])
```

List of dicts describing the sourced Vim scripts.

getstacktrace

```
getstacktrace()
```

List describing the current Vimscript call stack.

gettabinfo

```
gettabinfo([ {tabnr} ])
```

List of dicts with information about tab pages.

gettabvar

```
gettabvar({tabnr}, {varname} [, {def}])
```

Tab-local variable {varname} of tab {tabnr}.

gettabwinvar

```
gettabwinvar({tabnr}, {winnr}, {varname} [, {def}])
```

Window-local variable in window {winnr} of tab {tabnr}.

gettagstack

```
gettagstack([winnr])
```

Dict describing the tag stack of window {winnr}.

getwininfo

```
getwininfo([winid])
```

List of dicts with information about windows.

getwinpos

```
getwinpos([timeout])
```

[x, y] pixel position of the Vim window.

getwinposx

```
getwinposx()
```

X pixel position of the left of the Vim window, else -1 (stub in standalone).

getwinposy

```
getwinposy()
```

Y pixel position of the top of the Vim window, else -1 (stub in standalone).

getwinvar

```
getwinvar({nr}, {varname} [, {def}])
```

Window-local variable/option {varname} of window {nr}.

has

```
has({feature} [, {check}])
```

1 if Vim supports {feature}, else 0.

hasmapto

```
hasmapto({what} [, {mode} [, {abbr}])
```

1 if a mapping exists whose rhs contains {what}.

highlight_exists

```
highlight_exists({name})
```

1 if highlight group {name} is defined (alias of hlexists()).

histadd

```
histadd({history}, {item})
```

Add {item} to the {history} list (cmd/search/expr/input).

histdel

```
histdel({history} [, {item}])
```

Delete entries from the {history} list.

histget

```
histget({history} [, {index}])
```

Entry {index} from the {history} list.

histnr

```
histnr({history})
```

Index of the most recent entry in the {history} list.

hlID

```
hlID({name})
```

Numeric ID of highlight group {name} (0 if undefined).

hlexists

```
hlexists({name})
```

1 if highlight group {name} is defined, else 0 (stub in standalone).

id

```
id({expr})
```

Unique String identifier for the container or value {expr}.

input

```
input({prompt} [, {text} [, {completion}]]])
```

Prompt for and return a line of input from the user.

inputdialog

```
inputdialog({prompt} [, {text} [, {cancelreturn}]]])
```

Like input() but using a GUI dialog when available.

inputlist

```
inputlist({textlist})
```

Prompt the user to pick a numbered entry from {textlist}.

inputrestore

```
inputrestore()
```

Restore typeahead previously saved with inputsave() (stub in standalone).

inputsave

```
inputsave()
```

Save and clear pending typeahead so input() can be used (stub in standalone).

inputsecret

```
inputsecret({prompt} [, {text}])
```

Like input() but the typed text is hidden.

intercept

```
intercept({kind}, {pattern}, {code})
```

Register AOP advice ({kind} = before|after|around) on user-function calls matching {pattern}; returns the intercept ID (vimlrs extension).

intercept_proceed

```
intercept_proceed()
```

From an around advice, run the original intercepted function and return its value (vimlrs extension).

interrupt

```
interrupt()
```

Interrupt script execution as if CTRL-C were pressed (stub in standalone).

map

```
map({expr1}, {expr2})
```

Replace each item of List/Dict/Blob/String {expr1} with {expr2}; in place.

maparg

```
maparg({name} [, {mode} [, {abbr} [, {dict}]]])
```

rhs (or Dict) of the mapping of {name} in {mode}.

mapcheck

```
mapcheck({name} [, {mode} [, {abbr}]])
```

rhs of a mapping that {name} could match, else empty.

maplist

```
maplist([{abbr}])
```

List of dicts describing all mappings.

mapset

```
mapset({mode}, {abbr}, {dict})
```

Restore a mapping from a maparg() dict.

matchadd

```
matchadd({group}, {pattern} [, {priority} [, {id} [, {dict}]]])
```

Highlight {pattern} with {group}; returns the match id.

matchaddpos

```
matchaddpos({group}, {pos} [, {priority} [, {id} [, {dict}]]])
```

Highlight positions {pos} with {group}; returns the match id.

matcharg

```
matcharg({nr})
```

[{pattern}, {group}] for the {nr}'th :match command.

matchdelete

```
matchdelete({id} [, {win}])
```

Delete the match with {id}.

menu_get

```
menu_get({path} [, {modes}])
```

List of dicts describing the menus under {path}.

menu_info

```
menu_info({name} [, {mode}])
```

Dict of information about menu {name}.

mode

```
mode([ {expr} ])
```

Current mode as a short String ('n', 'i', 'v', ...).

preinserted

```
preinserted()
```

Text inserted by the current 'completefunc'/preinsert, if any.

prompt_appendbuf

```
prompt_appendbuf({buf}, {expr})
```

Append {expr} to prompt-buffer {buf} above the prompt (stub in standalone).

prompt_getinput

```
prompt_getinput({buf})
```

Current input line in prompt-buffer {buf} (stub in standalone).

prompt_getprompt

```
prompt_getprompt({buf})
```

Prompt string of prompt-buffer {buf} (stub in standalone).

prompt_setcallback

```
prompt_setcallback({buf}, {expr})
```

Set the callback invoked on Enter in prompt-buffer {buf} (stub in standalone).

prompt_setinterrupt

```
prompt_setinterrupt({buf}, {expr})
```

Set the callback invoked on CTRL-C in prompt-buffer {buf} (stub in standalone).

prompt_setprompt

```
prompt_setprompt({buf}, {text})
```

Set the prompt of prompt-buffer {buf} to {text} (stub in standalone).

pum_getpos

```
pum_getpos()
```

Dict describing the popup menu position, or empty if hidden.

pumvisible

```
pumvisible()
```

1 if the Insert-mode popup menu is visible, else 0 (stub in standalone).

reg_executing

```
reg_executing()
```

Name of the register being executed (replayed), or empty (stub in standalone).

reg_recorded

```
reg_recorded()
```

Name of the register last recorded into, or empty (stub in standalone).

reg_recording

```
reg_recording()
```

Name of the register being recorded into, or empty (stub in standalone).

screenattr

```
screenattr({row}, {col})
```

Highlight attribute id of the screen cell at {row},{col} (stub in standalone).

screenchar

```
screenchar({row}, {col})
```

Codepoint of the character on the screen at {row},{col} (stub in standalone).

screenchars

```
screenchars({row}, {col})
```

List of codepoints (incl. composing) at screen cell {row},{col}.

screencol

```
screencol()
```

Current cursor screen column (stub in standalone).

screenpos

```
screenpos({winid}, {lnum}, {col})
```

Screen position of buffer position {lnum},{col}.

screenrow

```
screenrow()
```

Current cursor screen row (stub in standalone).

screenstring

```
screenstring({row}, {col})
```

String of the character(s) at screen cell {row},{col} (stub in standalone).

search

```
search({pattern} [, {flags} [, {stopline} [, {timeout} [, {skip}]]]])
```

Search for {pattern}; returns the matching line number, else 0.

searchcount

```
searchcount([options])
```

Dict of search match counts for the last/used pattern.

searchdecl

```
searchdecl({name} [, {global} [, {thisblock}]]])
```

Move to the declaration of {name}; 0 if found (stub in standalone).

searchpair

```
searchpair({start}, {middle}, {end} [, {flags} ...])
```

Search for the matching {end} of a nested {start}/{end} pair.

searchpairpos

```
searchpairpos({start}, {middle}, {end} [, {flags} ...])
```

Like searchpair() but returns the [line, col] of the match.

searchpos

```
searchpos({pattern} [, {flags} [, {stopline} [, {timeout} [, {skip}]]]])
```

Like search() but returns the [line, col] of the match.

setcellwidths

```
setcellwidths({list})
```

Override the display cell width of character ranges from {list}.

setcharpos

```
setcharpos({expr}, {list})
```

Like setpos() but the column in {list} is a character index.

setcharsearch

```
setcharsearch({dict})
```

Set the last character-search state from {dict} (stub in standalone).

setcmdline

```
setcmdline({str} [, {pos}])
```

Set the command line to {str} with the cursor at {pos}.

setcmdpos

```
setcmdpos({pos})
```

Set the cursor position in the command line to {pos}.

setcursorcharpos

```
setcursorcharpos({lnum}, {col} [, {off}])
```

Move the cursor to character position {lnum},{col}.

setloclist

```
setloclist({nr}, {list} [, {action} [, {what}]])
```

Set the location list for window {nr}.

setmatches

```
setmatches({list} [, {win}])
```

Restore matches from a List produced by getmatches().

setpos

```
setpos({expr}, {list})
```

Set cursor/mark position {expr} from [bufnum, lnum, col, off].

setqflist

```
setqflist({list} [, {action} [, {what}]])
```

Set the quickfix list.

setreg

```
setreg({regname}, {value} [, {options}])
```

Set register {regname} to {value} with optional type {options}.

settabvar

```
settabvar({tabnr}, {varname}, {val})
```

Set tab-local variable {varname} of tab {tabnr} (stub in standalone).

settabwinvar

```
settabwinvar({tabnr}, {winnr}, {varname}, {val})
```

Set a window-local variable in window {winnr} of tab {tabnr} (stub in standalone).

settagstack

```
settagstack({winnr}, {dict} [, {action}])
```

Set the tag stack of window {winnr} (stub in standalone).

setwinvar

```
setwinvar({nr}, {varname}, {val})
```

Set window-local variable/option {varname} of window {nr} (stub in standalone).

shiftwidth

```
shiftwidth([{col}])
```

Effective 'shiftwidth' value (resolving 0 to 'tabstop').

sign_define

```
sign_define({name} [, {dict}])
```

Define sign {name} with attributes; 0 on success.

sign_getdefined

```
sign_getdefined([{name}])
```

List of defined signs (or just {name}).

sign_getplaced

```
sign_getplaced([{buf} [, {dict}])
```

List of signs placed in a buffer.

sign_jump

```
sign_jump({id}, {group}, {buf})
```

Jump to sign {id} of {group} in buffer {buf}.

sign_place

```
sign_place({id}, {group}, {name}, {buf} [, {dict}])
```

Place sign {name} in buffer {buf}; returns the sign id.

sign_placelist

```
sign_placelist({list})
```

Place a List of signs at once; List of ids.

sign_undefine

```
sign_undefine([ {name} ])
```

Undefine sign {name} (or all signs).

sign_unplace

```
sign_unplace({group} [, {dict}])
```

Remove placed sign(s) in {group}.

sign_unplacelist

```
sign_unplacelist({list})
```

Remove a List of placed signs.

spellbadword

```
spellbadword([ {sentence} ])
```

[badword, type] of the first misspelled word at/after the cursor.

spellsuggest

```
spellsuggest({word} [, {max} [, {capital} ]])
```

List of spelling suggestions for {word}.

state

```
state([ {what} ])
```

String of characters describing Vim's current busy/blocking state (stub in standalone).

synID

```
synID({lnum}, {col}, {trans})
```

Syntax ID at line {lnum}, column {col} (stub in standalone).

synIDattr

```
synIDattr({synID}, {what} [, {mode}])
```

Attribute {what} of syntax ID {synID} (stub in standalone).

synIDtrans

```
synIDtrans({synID})
```

Translated (effective) syntax ID for {synID} (stub in standalone).

synconcealed

```
synconcealed({lnum}, {col})
```

Info about concealed text at line {lnum}, column {col}.

synstack

```
synstack({lnum}, {col})
```

List of syntax IDs stacked at line {lnum}, column {col}.

tabpagebuflist

```
tabpagebuflist([{arg}])
```

List of buffer numbers shown in a tab page (stub in standalone).

tabpagenr

```
tabpagenr([{arg}])
```

Number of the current (or last) tab page (stub in standalone).

tabpagewinnr

```
tabpagewinnr({tabnr} [, {arg}])
```

Number of the current window in tab {tabnr} (stub in standalone).

tagfiles

```
tagfiles()
```

List of tags files used by the current buffer.

taglist

```
taglist({expr} [, {filename}])
```

List of tags matching the regexp {expr}.

test_garbagecollect_now

```
test_garbagecollect_now()
```

Test helper: run the garbage collector immediately (stub in standalone).

test_write_list_log

```
test_write_list_log({fname})
```

Test helper: write the internal list-allocation log to {fname} (stub in standalone).

timer_info

```
timer_info([id])
```

List of dicts describing active timers.

timer_pause

```
timer_pause({timer}, {paused})
```

Pause or unpause timer {timer} (stub in standalone).

timer_start

```
timer_start({time}, {callback} [, {options}])
```

Start a timer firing {callback} after {time} ms; returns its id (stub in standalone).

timer_stop

```
timer_stop({timer})
```

Stop timer {timer} (stub in standalone).

timer_stopall

```
timer_stopall()
```

Stop all timers (stub in standalone).

visualmode

```
visualmode([expr])
```

Last Visual mode used ('v', 'V', or CTRL-V) (stub in standalone).

wait

```
wait({timeout}, {condition} [, {interval}])
```

Wait until {condition} is true or {timeout} ms elapse (stub in standalone).

wildmenu

```
wildmenu()
```

1 if the wildmenu command-line completion menu is active (stub in standalone).

wildtrigger

```
wildtrigger()
```

Trigger command-line ('wildchar') completion programmatically (stub in standalone).

win_execute

```
win_execute({id}, {command} [, {silent}])
```

Execute {command} as if in window {id}, returning its output (stub in standalone).

win_findbuf

```
win_findbuf({bufnr})
```

List of window-IDs that display buffer {bufnr}.

win_getid

```
win_getid([{win} [, {tab}])
```

window-ID of window {win} in tab {tab} (stub in standalone).

win_gettype

```
win_gettype([{nr}])
```

Type of window {nr} ('popup', 'command', 'autocmd', ...).

win_gotoid

```
win_gotoid({id})
```

Go to the window with window-ID {id}; 1 on success (stub in standalone).

win_id2tabwin

```
win_id2tabwin({id})
```

[tabnr, winnr] of window-ID {id}.

win_id2win

```
win_id2win({id})
```

Window number of window-ID {id}, else 0 (stub in standalone).

win_move_separator

```
win_move_separator({nr}, {offset})
```

Move the vertical separator of window {nr} by {offset} (stub in standalone).

win_move_statusline

```
win_move_statusline({nr}, {offset})
```

Move the status line of window {nr} by {offset} (stub in standalone).

win_screenpos

```
win_screenpos({nr})
```

[row, col] screen position of the top-left of window {nr}.

win_splitmove

```
win_splitmove({nr}, {target} [, {options}])
```

Move window {nr} to split alongside window {target} (stub in standalone).

wincol

```
wincol()
```

Cursor's screen column within its window (stub in standalone).

winheight

```
winheight({nr})
```

Height (in lines) of window {nr}, else -1 (stub in standalone).

winlayout

```
winlayout([{{tabnr}}])
```

Nested List describing the window layout of a tab page.

winline

```
winline()
```

Cursor's screen row within its window (stub in standalone).

winnr

```
winnr([{arg}])
```

Number of the current (or related) window (stub in standalone).

winrestcmd

```
winrestcmd()
```

Ex command string that restores the current window sizes (stub in standalone).

winrestview

```
winrestview({dict})
```

Restore the window view from a winsaveview() dict (stub in standalone).

winsaveview

```
winsaveview()
```

Dict capturing the current window view (cursor, scroll).

winwidth

```
winwidth({nr})
```

Width (in columns) of window {nr}, else -1 (stub in standalone).

wordcount

```
wordcount()
```

Dict of byte/char/word counts for the buffer and cursor.

Ex Commands

7 entries

echo

:echo {expr} — evaluate and display {expr} (space-separated).

echon

:echon {expr} — like :echo but without a trailing newline.

echomsg

:echomsg {expr} — display {expr} and save it in the message history.

let

:let {var} = {expr} — assign the value of {expr} to {var}.

call

:call {func}({args}) — call a function and discard its return value.

eval

:eval {expr} — evaluate {expr} for its side effects.

Intercept

:Intercept before|after|around {pattern} { code } — register AOP advice on user-function calls; also :Intercept list | remove {id} | clear | proceed (vimlrs extension).

Options

15 entries

ignorecase

```
:set ignorecase (ic) - Bool, default 0
```

Ignore case in search patterns.

smartcase

```
:set smartcase (scs) - Bool, default 0
```

Override 'ignorecase' when the pattern has uppercase letters.

magic

```
:set magic (magic) - Bool, default 1
```

Use 'magic' regexp special-character meanings.

expandtab

```
:set expandtab (et) - Bool, default 0
```

Insert spaces instead of a <Tab>.

number

```
:set number (nu) - Bool, default 0
```

Show line numbers in the left margin.

relativenumber

```
:set relativenumber (rnu) - Bool, default 0
```

Show line numbers relative to the cursor.

wrap

```
:set wrap (wrap) - Bool, default 1
```

Wrap long lines to fit the window width.

hlsearch

```
:set hlsearch (hls) - Bool, default 0
```

Highlight all matches of the last search pattern.

incsearch

```
:set incsearch (is) - Bool, default 0
```

Show the match for the search pattern as it is typed.

autoindent

```
:set autoindent (ai) - Bool, default 0
```

Copy the indent of the current line to a new line.

tabstop

```
:set tabstop (ts) - Number, default 8
```

Number of spaces a <Tab> counts for.

shiftwidth

```
:set shiftwidth (sw) - Number, default 8
```

Number of spaces used for each step of (auto)indent.

softtabstop

```
:set softtabstop (sts) - Number, default 0
```

Number of spaces a <Tab> counts for while editing.

textwidth

```
:set textwidth (tw) - Number, default 0
```

Maximum text width; longer lines are broken (0 = off).

scrolloff

```
:set scrolloff (so) - Number, default 0
```

Minimum lines kept above and below the cursor.

Special v: Variables

3 entries

v:true

Boolean true; numeric value 1, string value "v:true".

v:false

Boolean false; numeric value 0, string value "v:false".

v:null

Null value; behaves like an empty string in most contexts.